

CoreCLR vs Mono en .NET MAUI

Notas de trabajo para un post técnico sobre el impacto del runtime en el peso del APK de release en Android.

Runtimes: Mono / CoreCLR Plataforma: Android Release

Nota 1 — ¿Por qué el APK con CoreCLR pesa más?

Al compilar .NET MAUI para Android en Release, se elige entre Mono (runtime histórico) y CoreCLR (runtime moderno portado a mobile).

Razones del mayor tamaño

Runtime más pesado por diseño

CoreCLR fue construido para desktop/server. Mono fue diseñado para dispositivos embebidos.

AOT genera .so más grandes

CoreCLR produce archivos nativos de mayor tamaño. Mono usa AOT más compacto.

Trimming menos maduro

El linker no es igual de agresivo con CoreCLR en mobile.

Mayor porción del BCL

CoreCLR incluye más BCL en el bundle; reducible con trimming agresivo.

Cómo reducir el tamaño con CoreCLR

```
<PublishTrimmed>true</PublishTrimmed>
<TrimMode>link</TrimMode>
<AndroidCreatePackagePerAbi>true</AndroidCreatePackagePerAbi>
<AndroidUseAapt2>true</AndroidUseAapt2>
<AndroidEnableAssemblyCompression>true</AndroidEnableAssemblyCompression>
```

Conclusión

CoreCLR prioriza rendimiento sobre tamaño. Microsoft está trabajando en reducir el peso del APK.

Nota 2 — Benchmark: Lista virtual con 10.000 items

BenchmarkListPage carga 10.000 items desde JSON (~1.5 MB) y mide 4 fases.

Fase	Operación
1. JSON deserialize	JsonSerializer.Deserialize<List<BenchmarkRawDto>>()
2. LINQ map + sort	Proyección DTO + OrderByDescending(score)
3. Date parse + format	DateTime.Parse + ToString × 10k
4. Bind collection	new ObservableCollection<BenchmarkItem>(list)

Archivos: Models/BenchmarkItem.cs, BenchmarkRawDto.cs, BenchmarkListViewModel.cs, BenchmarkListPage.xaml

Nota 3 — CoreCLR es estricto con coerción de tipos en bindings

Con x:DataType y bindings compilados, el tipo del modelo debe coincidir exactamente con el del control.

```
<DataTemplate x:DataType="models:BenchmarkItem">
```

```
<Label Text="{Binding Score}" />
</DataTemplate>
```

Mono

Conversión int→string implícita. No explota.

CoreCLR

Asignación int→string directa → InvalidCastException en TargetInvocationException.

Fix: StringFormat o exponer ScoreText como string desde el ViewModel.

Nota 4 — Benchmark: Animación de partículas con SkiaSharp

BenchmarkSkiaPage: partículas con trail en SKCanvasView. Mide physics ms y draw ms por frame.

Métrica	Descripción	Por qué importa
FPS	Frames/seg del frame actual	Referencia instantánea
avg FPS	Promedio últimos 120 frames	Rendimiento sostenido
min FPS	Mínimo histórico	GC pauses / JIT stalls
physics ms	Loop C# por frame	Rendimiento JIT
draw ms	Skia DrawCircle × N × 21	Interop .NET→Skia

```
dotnet build -f net11.0-android -c Release -p:UseMono=false # CoreCLR
dotnet build -f net11.0-android -c Release -p:UseMono=true  # Mono
```

Protocolo: Start → 30s estabilización JIT → anotar avg/min FPS, physics ms, draw ms → repetir en la otra app.

Métrica	CoreCLR	Mono
100 part — avg FPS	—	—
100 part — min FPS	—	—
100 part — physics ms	—	—
100 part — draw ms	—	—
200 part — avg FPS	—	—
200 part — min FPS	—	—
200 part — physics ms	—	—
200 part — draw ms	—	—
500 part — avg FPS	—	—
500 part — min FPS	—	—
500 part — physics ms	—	—
500 part — draw ms	—	—

Esperado: physics ms CoreCLR más rápido; min FPS Mono más estable; draw ms similar.

Pendientes para el post

- AAB vs APK en Play Store
- ¿Vale la pena el peso extra de CoreCLR?
- Cuándo conviene cada runtime
- Referencias docs Microsoft MAUI

Última actualización: 2026-07-08